

Why Rust streaming PCA currently trails Python

EffDim kernel investigation — RTX 6000 Blackwell host, 24 July 2026

Executive finding

The initial result was real, but “Python versus Rust” was not the most useful description of it. The benchmark primarily compared **multithreaded OpenBLAS** behind NumPy with a **single-threaded ndarray::dot** path in Rust.

Replacing the Rust chunk multiplication with faer’s parallel GEMM reduced the $100k \times 1024$ streaming run from **5.00 s to 2.09 s** at 16 threads. That is a **2.39× Rust speedup**, but NumPy/OpenBLAS remained faster at **0.76 s**. The kernel choice therefore explains a substantial part of the gap, not all of it.

Enabling ndarray’s BLAS feature and linking Rust directly to OpenBLAS improved the same $100k \times 1024$ path further to **1.30 s**. This is **3.84× faster** than the original Rust path and narrows Python’s lead to **1.77×**. OpenBLAS is the best tested Rust kernel for the large matrix, although faer remains slightly faster on the medium DESI matrix.

The similarity between regular Rust PCA and the original Rust streaming time is coincidental. The two paths do different work and encounter different bottlenecks. Streaming PCA is principally a memory-saving design; it is not inherently faster.

Controlled experiment

All methods used the same float64 UniverseTBD arrays, 4,096-row chunks, and three timed repetitions; the best repetition is reported. threadpoolctl restricted NumPy/OpenBLAS to the stated thread count. The optimized Rust variant passed the same count to faer’s Rayon backend. Each method ran in an isolated worker; peak RSS is the maximum total resident memory sampled every 5 ms, including the interpreter, memory-mapped input pages, and temporaries. PCA-95 is the minimum number of components needed to explain at least 95% of the variance.

The original rust_ndarray implementation does not use a threaded BLAS backend, so its 1-thread and 16-thread rows are expected to be nearly equal.

100,000 × 1,024 — Legacy Survey DINOv3

Implementation	Threads	Time (s)	PCA-95 dimension	Peak RSS (MiB)
Python	1	2.273	110	917.8
NumPy/OpenBLAS				
Rust	1	4.979	110	877.0
ndarray::dot				
Rust faer GEMM	1	4.889	110	880.9
Python	16	0.757	110	921.2
NumPy/OpenBLAS				
Rust	16	5.000	110	877.4
ndarray::dot				
Rust faer GEMM	16	2.091	110	881.1

20,465 × 768 — DESI DINOv3

Implementation	Threads	Time (s)	PCA-95 dimension	Peak RSS (MiB)
Python	1	0.269	64	234.7
NumPy/OpenBLAS				
Rust	1	0.564	64	189.3
ndarray::dot				

Implementation	Threads	Time (s)	PCA-95 dimension	Peak RSS (MiB)
Rust faer GEMM	1	0.552	64	192.2
Python	16	0.090	64	236.0
NumPy/OpenBLAS				
Rust	16	0.571	64	189.3
ndarray::dot				
Rust faer GEMM	16	0.191	64	190.8

1,496 × 1,024 — JWST DINOv3

Implementation	Threads	Time (s)	PCA-95 dimension	Peak RSS (MiB)
Python	1	0.103	145	81.9
NumPy/OpenBLAS				
Rust	1	0.208	145	75.7
ndarray::dot				
Rust faer GEMM	1	0.204	145	79.2
Python	16	0.056	145	83.3
NumPy/OpenBLAS				
Rust	16	0.212	145	75.6
ndarray::dot				
Rust faer GEMM	16	0.169	145	87.3

OpenBLAS follow-up

The follow-up enabled `ndarray = { features = ["blas"] }` and linked OpenBLAS 0.3.32 into the PyO3 extension. NumPy used its independently bundled OpenBLAS 0.3.33 build. Both libraries were restricted to the reported thread count by `threadpoolctl`.

Sixteen-thread comparison

Dataset	Python OpenBLAS (s)	Rust OpenBLAS (s)	Rust faer (s)	Rust OpenBLAS peak RSS (MiB)	PCA-95
JWST	0.055	0.228	0.162	87.1	145
DESI	0.090	0.220	0.187	198.0	64
Legacy Survey	0.737	1.303	1.810	888.6	110

Effect of threading on Rust OpenBLAS

Dataset	One thread (s)	Sixteen threads (s)	Parallel speedup
JWST	0.198	0.228	0.87×
DESI	0.486	0.220	2.21×
Legacy Survey	4.275	1.303	3.28×

OpenBLAS helps once the chunk products contain enough work. Its thread startup and synchronization overhead makes the small JWST case slower at 16 threads. On the 100k matrix it beats faer by 1.39×, uses 33.6 MiB less peak RSS than Python, and reproduces the same PCA-95 dimension.

One-million-row result

A direct $1,000,000 \times 768$ float64 benchmark used a seeded standard-normal matrix, 4,096-row chunks, 16 threads, and three repetitions. The 5.72 GiB input was memory-mapped and reused by isolated workers.

Implementation	Best time (s)	Samples/s	PCA-95 dimension	Peak RSS (MiB)
Python	3.437	290,992	728	5,961.6
NumPy/OpenBLAS				
Rust ndarray/OpenBLAS	4.154	240,707	728	5,937.6
Rust faer GEMM	6.354	157,380	728	5,946.1

Rust OpenBLAS is **1.21× slower** than Python at this scale, substantially closer than the 1.77× gap on the $100k \times 1,024$ real matrix. It is **1.53× faster** than faer. All methods agree on PCA-95 and their eigenvalues differ by less than 8.0×10^{-15} in absolute terms.

Peak RSS is dominated by resident pages from the 5.72 GiB memory-mapped input; it should not be read as PCA workspace alone. Rust OpenBLAS peaked 24.1 MiB below Python.

Why Python wins this benchmark

1. **NumPy delegates the dominant operation to OpenBLAS.** Each chunk computes `centered.T @ centered`, a dense matrix multiplication. OpenBLAS uses 16 CPU threads effectively on the benchmark host.
2. **The original Rust path was effectively single-threaded.** `ndarray::dot` was built without an equivalent threaded BLAS backend. Changing the external thread limit consequently had no measurable effect.
3. **The first faer implementation still pays conversion costs.** Each chunk is copied from the NumPy/ndarray row-major representation into a faer matrix. Mean calculation and the covariance merge are also serial.
4. **Rust still finishes with a general SVD.** The covariance is symmetric positive semidefinite, so a self-adjoint eigensolver is the more appropriate final kernel. NumPy uses `eigvalsh`; the current Rust experiment still calls faer’s general singular-value routine.
5. **Small cases expose fixed costs.** At 1,496 rows, decomposition and conversion overhead are a larger fraction of the run, so parallel GEMM provides only a modest improvement.

Why streaming Rust resembles regular Rust

In the earlier head-to-head run on the $100k \times 1,024$ array:

Implementation	Regular PCA (s)	Streaming PCA (s)	PCA-95 dimension	Worker peak RSS (MiB)
Python	23.163	0.932	110	9,766.1
Rust (ndarray streaming)	5.119	5.003	110	5,270.2

Those older memory values are whole-worker maxima spanning both PCA variants, so they should not be interpreted as path-specific peaks. The controlled tables above isolate every streaming method.

Regular Rust performs a decomposition on the tall centered data matrix. Streaming Rust performs roughly 25 chunked covariance products followed by a $1,024 \times 1,024$ decomposition. Their nearly equal totals do **not** imply that streaming has no effect: streaming avoids materializing another full centered matrix and bounds temporary working memory by chunk size. It simply traded the regular path’s decomposition cost for about the same amount of unoptimized covariance work.

Numerical correctness

All implementations agreed to near machine precision. Across the controlled runs, the maximum absolute eigenvalue difference was at most 2.49×10^{-14} ; the maximum error relative to the leading eigenvalue was at most 1.82×10^{-15} . The timing differences are therefore not caused by a lower-accuracy computation.

Recommendation

Use streaming covariance when bounded memory is required, but do not yet assume the Rust path is the fastest implementation. On this host, NumPy/OpenBLAS remains the performance baseline.

For the Rust production path, OpenBLAS is now the leading large-matrix option:

1. avoid copying each row-major chunk into a new faer matrix;
2. retain tuned OpenBLAS dgemm and evaluate a symmetric rank-k update (SYRK);
3. replace covariance SVD with a self-adjoint eigensolver;
4. parallelize or vectorize mean and covariance-merge loops;
5. profile accumulation, conversion, merge, and decomposition separately;
6. repeat with fixed CPU affinity and matched physical-core counts.

OpenBLAS cuts the original large-case Rust time by almost fourfold, but more layout and decomposition work is still needed to beat NumPy on this host.

Reproducibility

- Benchmark: `benchmarks/bench_streaming_pca_kernels.py`
- Raw data: `benchmark-results/streaming_pca_kernel_comparison.csv`
- Structured results: `benchmark-results/streaming_pca_kernel_comparison.json`
- OpenBLAS results: `benchmark-results/streaming_pca_openblas_comparison.csv`
- One-million-row results: `benchmark-results/streaming_pca_openblas_1m_x_768.csv`
- Rust experiment: `spectral_eigenvalues_streaming_faer`
- Host: AMD Ryzen Threadripper PRO 7975WX, RTX 6000 Blackwell workstation
- Timing excludes loading the `.npy` dataset and reports the best of three runs.
- Peak RSS is an absolute process maximum, not memory allocated solely by PCA.